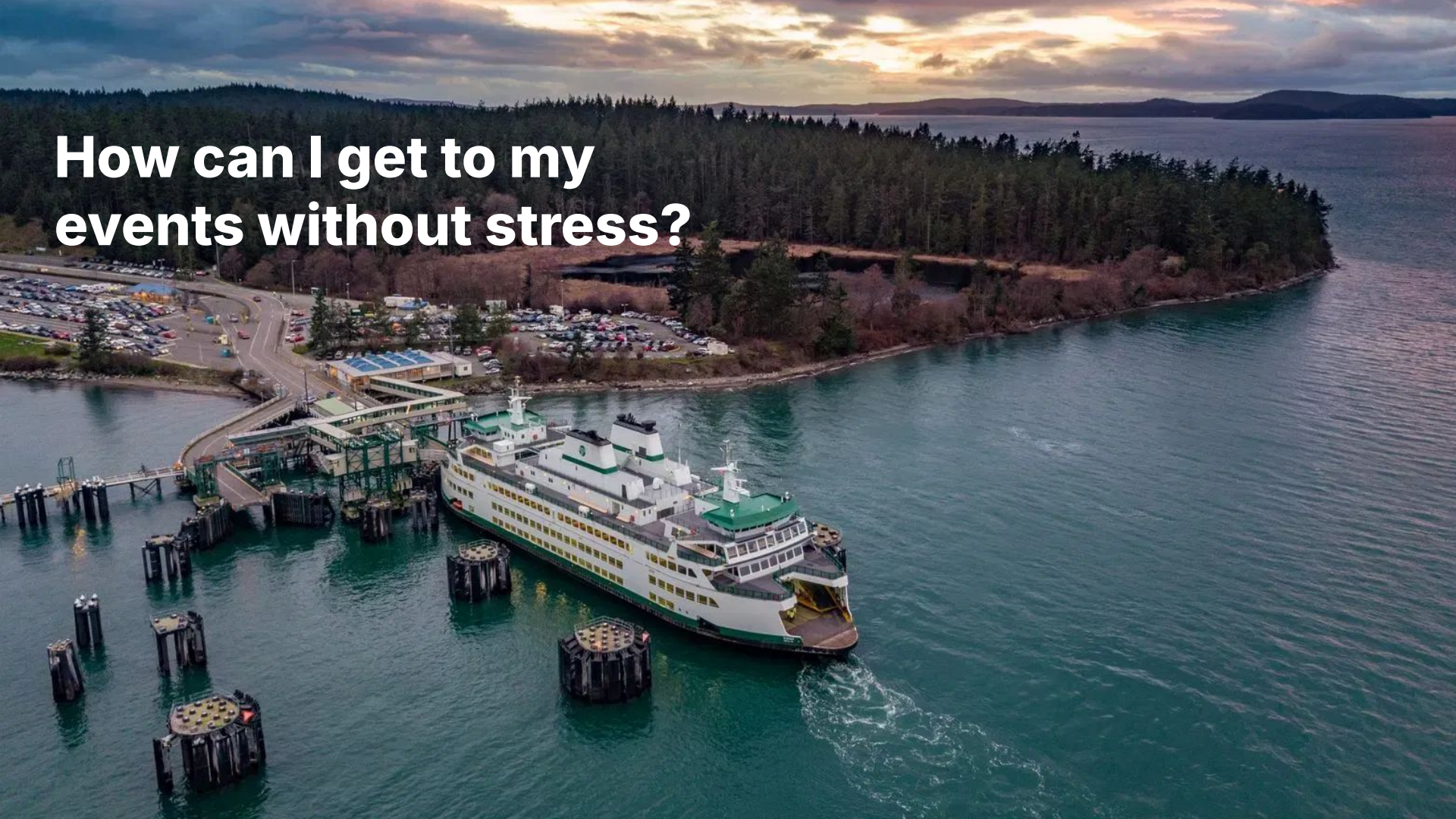
There and back again… but by ferry or I-5?

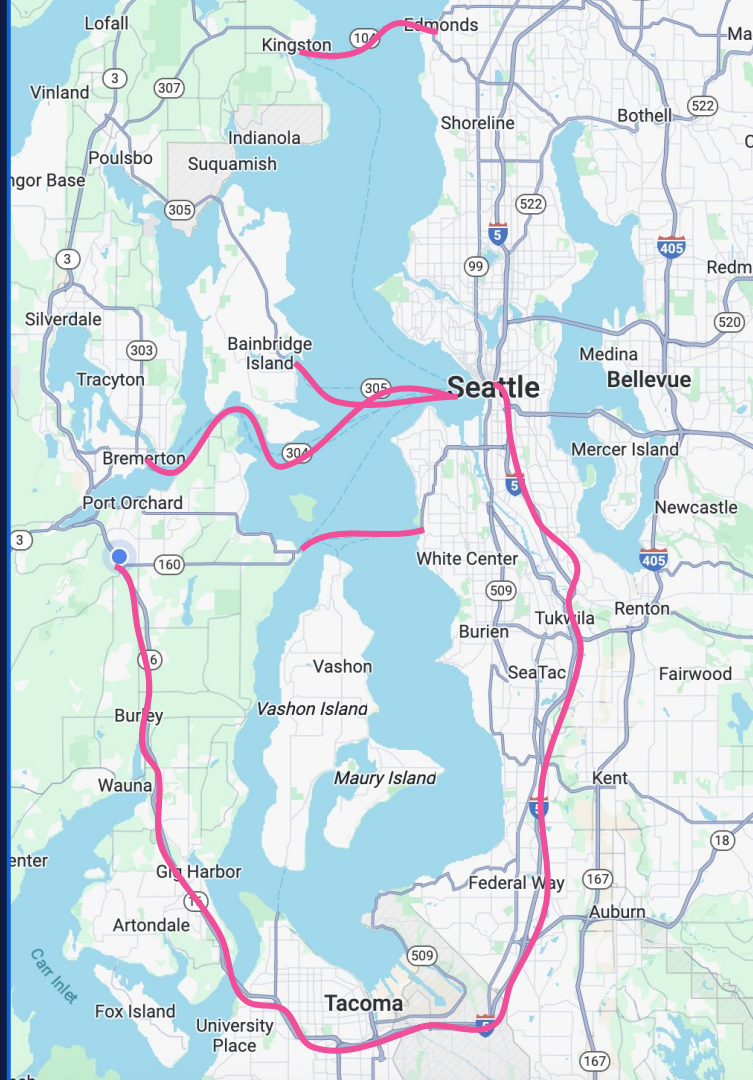
An exploration of Agentic AI components through the lens of a Pacific Northwest commuter

Justin Castilla



How can I get to my events without stress?

As an Olympic Peninsula resident, I have to find the best route to get to my meetups, concerts, dinners, etc.



Google Maps helps, but doesn’t consider schedules
elastic

How I built this solution

Python + FastMCP + Elasticsearch + LLM = Easier Commute with my own Agent

Step 1

Gather all of the information needed to make my commuting decisions

My Calendar

- Events
  - time
  - location
  - description
  - presenting?

WSDOT API

- Terminal locations
- Sailing Schedule
- Service Delays
- Cost Calculator

Google Maps API

- Drive time estimation
- Traffic monitoring

How I built this solution

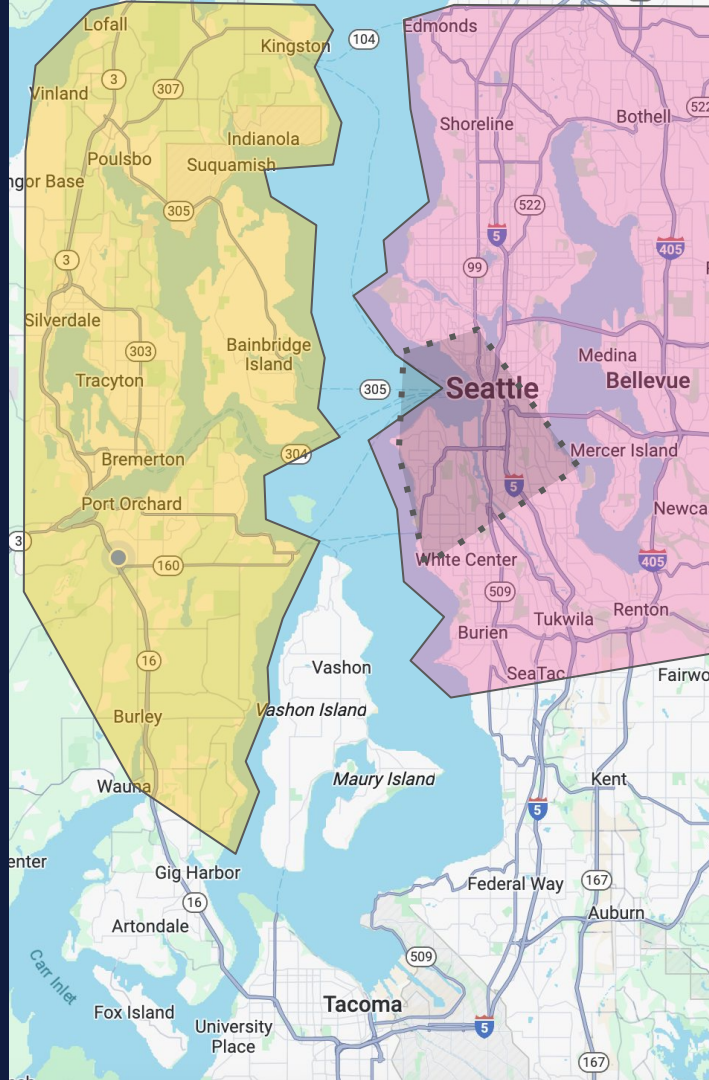
Python + FastMCP + Elasticsearch + LLM = Easier Commute?

Step 1

Gather all of the information needed to make my commuting decisions

Step 2

Establish a timetable of ferry routes and group them by regions and county



How I built this solution

Python + FastMCP + Elasticsearch + LLM = Easier Commute?

Step 1

Gather all of the information needed to make my commuting decisions

Step 2

Establish a timetable of ferry routes and group them by regions and county

Step 3

Write every single function I would need to determine drive times, ferry sailings, event creation and lookup

How I built this solution

Group functions logically into three MCP servers

Server	Tools
wsodt_server.py	find_nearest_terminals get_ferry_schedule get_todays_sailings get_ferry_fare get_drive_time estimate_total_travel generate_expense_estimate
events_write_server.py	create_event save_travel_plan
events_read_server.py	search_events (Elastic Agent Builder wrapper)

How I built this solution

Python + FastMCP + Elasticsearch + LLM = Easier Commute?

Step 1

Gather all of the information needed to make my commuting decisions

Step 2

Establish a timetable of ferry routes and group them by regions and county

Step 3

Write every single function I would need to determine drive times, ferry sailings, event creation and lookup

Step 4

Give an LLM access to these functions and provide **VERY** clear instructions

How I built this solution

Python + FastMCP + Elasticsearch + LLM = Easier Commute?

Python

Easiest language to use for fast prototyping of MCP servers, interacting with Elastic, and general hackery.

FastMCP

Creates a server of resources, tools, and prompts for the LLM to use

Elasticsearch

Agent Builder

Queries my events and expense data without a need for vector query knowledge

APIs

WDOT & Google Maps

How I built this solution

Python + FastMCP + Elasticsearch + LLM = Easier Commute?

Python

Easiest language to use for fast prototyping of MCP servers, interacting with Elastic, and general hackery.

FastMCP

Creates a server of resources, tools, and prompts for the LLM to use

Elasticsearch

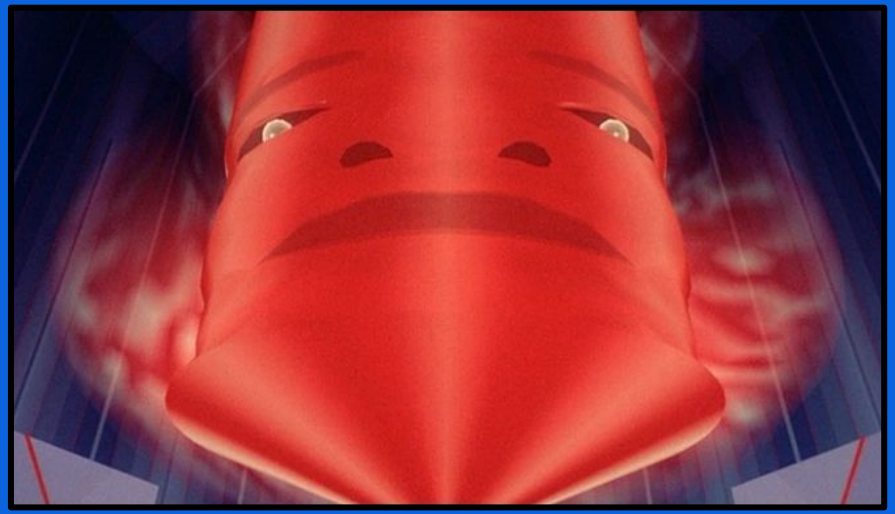
Hold my calendar events in a document store, with semantic and hybrid search at the ready

Docker

Everything contained in its own environment

MCP, like the MCP from Tron?

- A framework made by an Anthropic engineer with agreed upon communication protocols and schemas
- Creates ***resources***, ***tools***, ***prompts***, and other services available to LLMs to access and execute
- Allows LLMs to reach out and gather more information, take actions on behalf of users, and generally do more than just sit on their dated knowledge
- The backbone of Agentic AI



MCP (Master Control Program), from Disney's Tron, 1982

The three major components of MCP

Resources

Resources represent data or files that an MCP client can read.

```
@mcp.resource(
    uri="config://environment",
    name="Current Environment",
    description="Returns active deployment configuration",
)
def get_environment_config() -> dict:
    return {
        "region": os.getenv("AWS_REGION"),
        "feature_flags": load_flags(),
        "rate_limit_rpm": 1000,
    }
```

The three major components of MCP

Resources

Resource templates extend this concept

```
@mcp.resource("file://{path}")

def read_project_file(path: str) -> str:
    with open(path, encoding="utf-8") as:
        return open(path).read()
```

The three major components of MCP

Tools

Tools in MCPs transform regular Python functions into capabilities that LLMs can invoke during conversations.

When an LLM **decides** to use a tool:

1. It sends a request with parameters based on the tool's schema.
2. The MCP validates these parameters against your function's signature. Pydantic for the win!
3. The function executes with the validated inputs.
4. The result is returned to the LLM, which can use it in its response.

The three major components of MCP

Tools

```python
@mcp.tool(
    name="create_event",
    description="Create a new event in Elasticsearch."
)
def create_event(event_doc: dict) -> dict:
    resp = es.index(index=EVENT_INDEX, document=event_doc)
    return {"event_id": resp["_id"], "event": event_doc}
```

The three major components of MCP

Resources

	Resources	Tools
Analogy	File / document	Function call
Triggered by	Client requests it	LLM decides to call it
Side effects	None	Yes (API calls, DB writes, etc.)
Good for	Reference data, config, schemas	Actions, lookups, computations

Resources are the **read-only** data layer of MCP. Think of them as the difference between a file you can open and read vs. a button you can press.

The three major components of MCP

Prompts

Prompts are reusable message templates that help LLMs generate structured, purposeful responses.

When a client requests a prompt:

1. MCP finds the corresponding prompt definition.
2. If it has parameters, they are validated against your function signature.
3. Your function executes with the validated inputs.
4. The generated message is returned to the LLM to guide its response.
5. This allows you to define consistent, reusable templates that LLMs can use across different clients and contexts.

The three major components of MCP

Prompts

```
@mcp.prompt(
        name="get_there_with_a_buffer",
        description="Instructions for finding a route with buffer before event time"
)
 def get_there_by(origin, destination, event_time, buffer_minutes=10):
        return f"""
         1. The starting location is {origin}.
        2. Find the two nearest ferry terminals from {origin} and calculate drive times to each.
        3. For each ferry option, determine the arrival time at the destination terminal.
        4. Calculate the drive time from the destination terminal to {destination}.
        5. Also calculate a driving-only route from {origin} to {destination}.
        6. The target arrival time is {buffer_minutes} minutes before {event_time}.
        7. Subtract {buffer_minutes} minutes from {event_time} to get the latest acceptable arrival time.
        8. Return only the options that arrive by that target time, sorted by total travel time. """
```

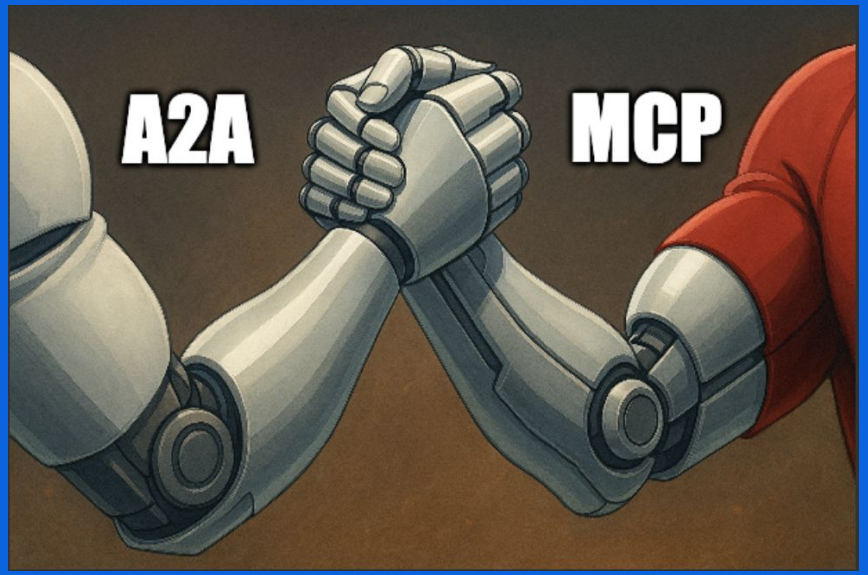
FastMCP, to make things fast and easy!

github.com/jlowin/fastmcp
https://gofastmcp.com/

- MCP is powerful but implementing it involves a lot of boilerplate
  a. server setup
  b. protocol handlers
  c. content types
  d. error management.
- FastMCP is high-level and Pythonic
- In most cases, decorating a function is all you need.
- Part of the MCP framework
- But also it's own project
- VERY active

MCP does <u>not</u> compete with Agentic AI

- MCP servers provide tools and means to empower the LLM with extended abilities
- LLM Agents are able to make decisions, direct workflows, and iterate on a problem
- LLM Agents can and **should** have MCP servers at their disposal



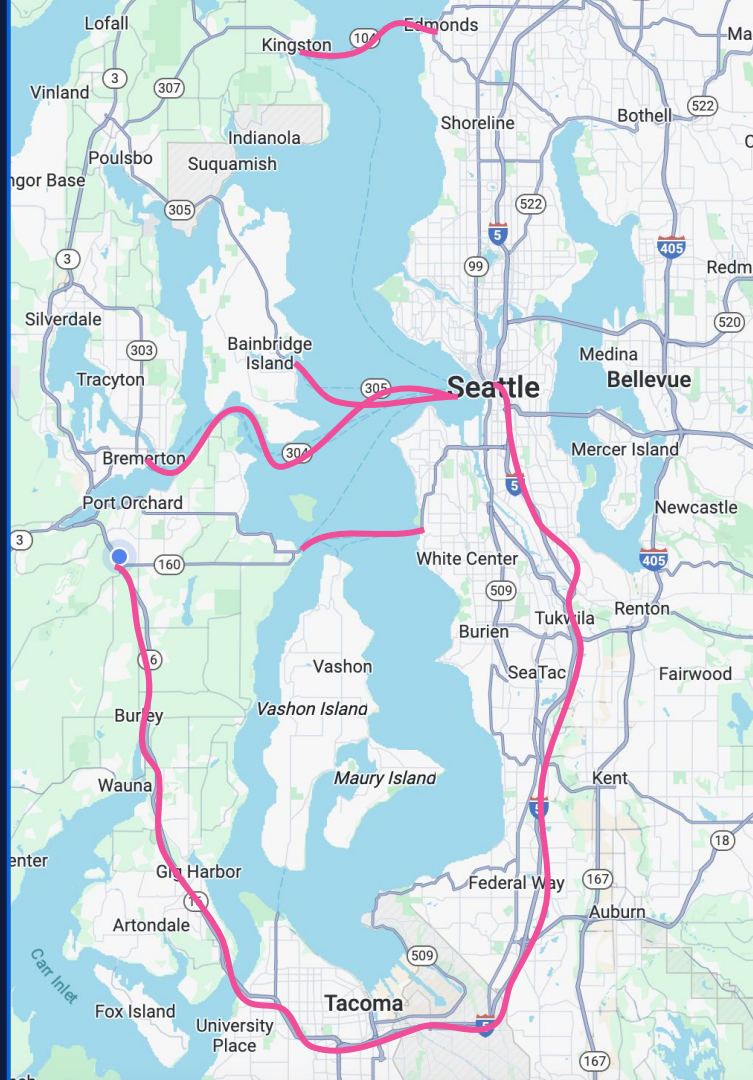
Check out the code:

https://github.com/justincastilla/kitsap_commute_MCP

elastic

Further work!

- Observability!
- Passenger only ferry integration
- Check for schedule disruptions
- Send driving directions to phone
- Create calendar reminders to start driving
- Store my chosen selections and prioritize them if they worked out well for me
- Create an multi-agent version (beyond MCP)



Thank you!

elastic | The Search AI Company